DIPLOMADO EN MACHINE LEARNING Y DATA SCIENCE AVANZADO · DEEP LEARNING

Tarea 5 · Modelos generativos

Traducción de texto a emojis con un Transformer codificador-decodificador



Guía de estudio: el problema, la teoría de fondo, el código explicado y la intervención

Módulo 5, unidad 5 · Universidad Nacional de Colombia, Facultad de Ingeniería

Notebook: `v2_M5U5_Tarea_5_Resuelto.ipynb`

Natalia Patricia Remolina Rodríguez · 7 de noviembre de 2026

✓ Calificación en UNCode: 100 %, 11 de 11 pruebas en 100

La tarea en una página

El problema

- Convertir una frase en inglés en una secuencia de emojis que conserve su sentido.
- Entrada y salida tienen longitudes distintas y "idiomas" distintos: es un problema **secuencia a secuencia**.
- Unas 10.000 parejas, en su mayoría intervenciones del Parlamento Europeo.

La solución

- Un **Transformer codificador-decodificador** construido pieza por pieza en Keras.
- El codificador lee la frase; el decodificador escribe los emojis uno a uno y consulta al codificador mediante **atención cruzada**.
- Se entrena con **teacher forcing** y una pérdida que ignora el relleno.

La intervención

- `preprocess` pasó de 80 a 100 al dejar los emojis **sin separar**.
- Codificador y decodificador pasaron de 0 a 100 al usar los **nombres de atributos** que inspecciona el calificador.
- Se añadió `dropout=rate` a todas las capas de atención.

El resultado

- Las 11 pruebas en 100 y envío oficial con **100 %**.
- La pérdida bajó de 8,03 a 3,48; la mejor validación fue 3,11 en la época 19 y después hubo sobreajuste.
- Ejemplo generado: 🇪🇺❌🚫, correcto en lo mecánico y parcial en el significado.

CÓMO LEER ESTA GUÍA

La guía avanza de lo general a lo concreto. La **parte I** plantea el problema. La **parte II** explica la teoría que hace falta para entender cada línea, con ejemplos numéricos pequeños. La **parte III** recorre el notebook celda por celda y sigue la forma de los tensores en cada paso. La **parte IV** cuenta qué falló, cómo se diagnosticó y qué se cambió. La **parte V** interpreta los resultados, y la **parte VI** cierra con preguntas de autoevaluación y un glosario.

Las cajas de colores tienen siempre el mismo sentido:

QUÉ SE PEDÍA
la instrucción del enunciado.

INTUICIÓN
la idea detrás del código.

INTERVENCIÓN
lo que hubo que cambiar para llegar al 100 %.

OJO
un detalle fácil de pasar por alto.

Contenido

Parte I · El problema

1. Qué pide la tarea
2. Cómo califica UNCode

Parte II · La teoría de fondo

3. De texto a números
4. La atención por dentro
5. El bloque Transformer
6. Cómo aprende y cómo genera

Parte III · La solución celda por celda

7. Preparación y datos
8. Punto 1 · preprocess
9. Punto 2 · vectorize
10. Punto 3 · codificador
11. Punto 4 · decodificador
12. Punto 5 · get_model
13. Punto 6 · train_model
14. Generación y envío

Parte IV · La intervención

15. El método de diagnóstico
16. Las tres intervenciones
17. Lecciones

Parte V · Resultados

18. Las pruebas
19. Curvas de entrenamiento
20. El ejemplo generado
21. Mejoras posibles

Parte VI · Para consolidar

22. Preguntas de autoevaluación
23. Glosario
24. Conclusiones

PARTE I

El problema

Qué hay que resolver, con qué datos y bajo qué reglas de calificación.

1. Qué pide la tarea

Dado un texto en inglés, el modelo debe producir una secuencia de emojis que capture su significado y su tono. El enunciado insiste en que no es una traducción palabra por palabra: el modelo tiene que comprender el contexto, identificar los conceptos clave y las emociones implícitas, y generar una secuencia coherente.

Texto en inglés	Emojis
The ratification and implementation of the updated ILO conventions vote	✅🗳️
Plus we agreed to delete point a	❌
Parliament agreed to accept the oral amendment	✅💬

Por qué no basta un diccionario de palabra a emoji

- **Las longitudes no coinciden.** Doce palabras se convierten en dos emojis. No hay una correspondencia uno a uno.
- **El significado depende del conjunto.** En «agreed to delete point a», el emoji ❌ no sale de ninguna palabra aislada: sale de la idea de eliminar algo.
- **El orden importa en la salida.** Los emojis se escriben uno tras otro y cada uno depende de los anteriores.

Este tipo de problema se llama **secuencia a secuencia** o *seq2seq*. Entra una secuencia de longitud variable y sale otra, de otro vocabulario y otra longitud. Es la misma estructura de la traducción automática entre idiomas, y por eso la arquitectura es la misma: un **Transformer codificador-decodificador**, como el del artículo *Attention Is All You Need* de Vaswani y colaboradores, 2017.

Los seis puntos calificados

Punto	Función	Qué debe hacer	Entra → sale
1	`preprocess`	Limpiar el inglés y marcar inicio y fin en ambas columnas	DataFrame → DataFrame
2	`vectorize`	Construir los vocabularios y convertir texto en enteros	DataFrame → 2 vectorizadores
3	`get_EncoderLayer`	Capa personalizada con el codificador	`(lote, 26)` → `(lote, 26, 128)`
4	`get_DecoderLayer`	Capa personalizada con el decodificador	`(lote, 25)` → `(lote, 25, 5000)`

5	`get_model`	Unir ambas capas en un `keras.Model`	2 entradas → probabilidades
6	`train_model`	Compilar y entrenar desfasando la salida un token	modelo → `history`

2. Cómo califica UNCode

Cada punto tiene una celda de prueba local y una o dos celdas `grader.run_test(...)`. El calificador no se limita a comprobar que el código corra: ejecuta la función con sus propias entradas y compara contra una solución de referencia en tres niveles.

Nivel	Qué compara	Consecuencia práctica
Forma	Las dimensiones de la salida, por ejemplo `(10, 26, 128)`	Un +1 de más o de menos falla
Valores exactos	Los números de la salida, que dependen de las semillas y del orden en que se crean las capas	Dos arquitecturas equivalentes pero distintas dan números distintos
Estructura interna	Qué atributos tiene la capa, de qué tipo son y con qué parámetros se crearon	El **nombre** de cada atributo es parte de la respuesta

> **OJO**
> El tercer nivel explica la mayor parte de la intervención descrita en la parte IV. Un codificador correcto en lo matemático obtuvo 0 puntos solo porque sus atributos tenían otros nombres.

PARTE II

La teoría de fondo

Lo mínimo que hay que entender para que cada línea del código tenga sentido.

3. De texto a números

Una red neuronal solo opera con números. El primer trabajo es convertir cada frase en una fila de enteros de longitud fija. Eso ocurre en tres pasos.

Paso 1 · Tokens

Un **token** es la unidad mínima que ve el modelo. `TextVectorization` separa por espacios, así que en inglés cada palabra es un token. Se añaden dos tokens especiales: `[BOS]`, inicio de secuencia, y `[EOS]`, fin de secuencia. El decodificador necesita `[BOS]` para arrancar y aprende a producir `[EOS]` para detenerse.

Paso 2 · Vocabulario e identificadores

El vectorizador cuenta los tokens del entrenamiento y los ordena por frecuencia. Cada uno recibe un entero. Los cuatro primeros son fijos:

Id	Token	Por qué tiene ese lugar
0	relleno	Reservado por Keras
1	`[UNK]`	Reservado: token desconocido
2	`[EOS]`	Aparece en todas las frases
3	`[BOS]`	Aparece en todas las frases

Paso 3 · Relleno a longitud fija

Todas las filas se rellenan con ceros hasta medir 26, para poder apilarlas en una matriz. Así queda la fila 0:

Inglés · 26 posiciones

`[BOS]=3` `the` `ratification` `and` `implementation` `of` `the` `updated` `ilo` `conventions` `vote` `[EOS]=2` `0` `0` … `0`

Emojis · 26 posiciones

`[BOS]=3` ✅📦 `[EOS]=2` `0` `0` `0` … `0`

Figura 1. La fila 0 tokenizada. Cada caja gris es relleno. En la columna de emojis, toda la cadena ✅📦 es un solo token porque no tiene espacios dentro.

De enteros a vectores: los embeddings

Un entero como 3 o 417 no dice nada sobre el significado. Un **embedding** es una tabla entrenable que asigna a cada id un vector de 128 números. Al principio son aleatorios; durante el entrenamiento se acomodan para que los tokens que cumplen funciones parecidas queden cerca.

La atención no distingue el orden de los tokens: para ella, una frase es un conjunto. Por eso se suma un segundo embedding, el de **posición**, que aprende un vector por cada lugar 0, 1, 2… El vector final de cada token es la suma de los dos:

INTUICIÓN

`x[i] = token_emb(id del token i) + pos_emb(i)`. El primer sumando dice «qué palabra es» y el segundo «dónde está».

4. La atención por dentro

La atención es el mecanismo central del Transformer. Permite que cada token construya su nueva representación mezclando información de los demás, con pesos que decide el propio modelo.

4.1 Consultas, claves y valores

Cada vector de entrada se proyecta en tres vectores con tres matrices entrenables:

- **Q, la consulta:** lo que el token está buscando.
- **K, la clave:** lo que cada token ofrece para ser encontrado.
- **V, el valor:** la información que cada token entrega si lo eligen.

INTUICIÓN

Es como buscar en una biblioteca. La consulta es la pregunta, las claves son los títulos de los libros y los valores son su contenido. Se compara la pregunta con cada título, se decide cuánto leer de cada libro y se arma la respuesta con esa mezcla.

La fórmula completa es:

$$\text{Atención}(Q, K, V) = \text{softmax}\left(Q \cdot K^{\intercal} / \sqrt{d} \right) \cdot V$$

1. `Q·Kᵀ` compara cada consulta con cada clave mediante un producto punto. Da una matriz de puntajes.
2. Dividir por `√d` evita que los puntajes crezcan demasiado cuando la dimensión es grande, lo que saturaría el softmax.
3. `softmax` convierte cada fila de puntajes en pesos positivos que suman 1.
4. Multiplicar por `V` produce, para cada token, el promedio ponderado de los valores.

4.2 Un ejemplo con números

Tres tokens con vectores de dimensión 2. Para simplificar, Q, K y V son iguales a la entrada X:

Token	X	Puntajes $Q \cdot K^{\intercal}/\sqrt{2}$			Pesos tras softmax			Salida = pesos·V
		a	b	c	a	b	c	
a	[1, 0]	0,71	0	0,71	0,40	0,20	0,40	[0,80, 0,60]
b	[0, 1]	0	0,71	0,71	0,20	0,40	0,40	[0,60, 0,80]
c	[1, 1]	0,71	0,71	1,41	0,25	0,25	0,50	[0,75, 0,75]

El token *a* se parece a sí mismo y a *c*, porque comparten la primera coordenada; por eso les da 0,40 a cada uno y solo 0,20 a *b*. Su salida `[0,80, 0,60]` ya no es solo «a»: incorpora algo de los otros dos. Eso es lo

que significa que un vector «tenga contexto».

4.3 Varias cabezas

Con `num_heads=2`, el cálculo se hace dos veces en paralelo, cada una con sus propias matrices Q, K y V. Cada cabeza puede especializarse en un tipo de relación distinto, por ejemplo una en la sintaxis y otra en el tema. Al final, las salidas se concatenan y una proyección las devuelve a 128 dimensiones.

4.4 Auto-atención y atención cruzada

Tipo	De dónde sale Q	De dónde salen K y V	Llamada en Keras
Auto-atención	De la misma secuencia	De la misma secuencia	`att(x, x)`
Atención cruzada	De los emojis, en el decodificador	Del inglés, la salida del codificador	`att(x, contexto)`

La atención cruzada es el puente entre las dos mitades: cada posición de emojis pregunta «¿a qué palabras del inglés debo prestar atención ahora?».

4.5 La máscara causal

Al entrenar, el decodificador recibe la secuencia completa de emojis. Si la posición 1 pudiera mirar la posición 2, vería la respuesta que debe predecir. La máscara causal lo impide: antes del softmax, los puntajes hacia el futuro se reemplazan por $-\infty$, así su peso queda en 0.

	[BOS]	e_1	e_2	e_3
[BOS]	✓	✗	✗	✗
e_1	✓	✓	✗	✗
e_2	✓	✓	✓	✗
e_3	✓	✓	✓	✓

Cada fila es una posición que pregunta y cada columna una posición que puede ser mirada. Solo se permite mirar hacia atrás: la matriz es **triangular inferior**.

Con el ejemplo numérico anterior y la máscara puesta, el token *a* solo puede mirarse a sí mismo y obtiene pesos `[1, 0, 0]`; *b* obtiene `[0,33, 0,67, 0]`; *c* no cambia, porque ya estaba al final.

En el código basta un argumento: `self_att(x, x, use_causal_mask=True)`.

5. El bloque Transformer

Alrededor de la atención hay tres piezas que se repiten en cada subbloque.

Pieza	Qué hace	Por qué hace falta
Conexión residual	Suma la entrada a la salida del subbloque: `x + atención(x)`	Si el subbloque no aporta nada, la información original sigue pasando. Facilita que el gradiente llegue a las primeras capas
Normalización de capa	Reescala cada vector para que tenga media 0 y varianza 1, con dos parámetros entrenables	Mantiene los valores en una escala estable de capa en capa

Feed-forward	Dos capas densas por posición: 128 → 32 con ReLU → 128	La atención solo mezcla; esta red transforma cada vector de forma no lineal
Dropout	Apaga al azar un 10 % de los valores durante el entrenamiento	Evita que el modelo dependa de neuronas concretas. En inferencia no actúa

El patrón de cada subbloque es siempre el mismo: `x = norm(x + dropout(subbloque(x)))`.

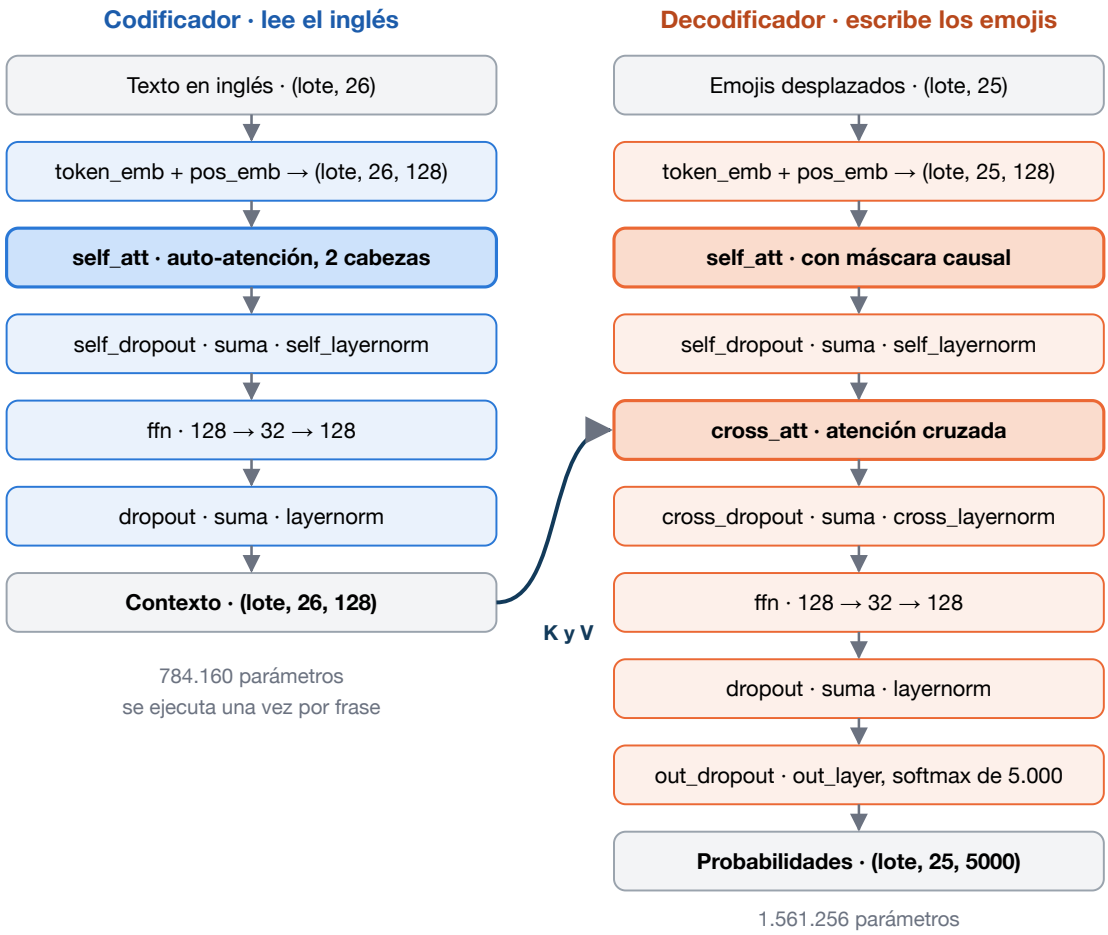


Figura 2. El modelo completo con los nombres de atributos que exige el calificador. El contexto del codificador entra al decodificador solo por la atención cruzada, como claves y valores; las consultas salen de los emojis.

6. Cómo aprende y cómo genera

6.1 Teacher forcing: el desfase de un token

Para entrenar, el decodificador recibe la secuencia correcta de emojis corrida una posición respecto al objetivo. En cada posición ve el token actual y los anteriores, y debe acertar el siguiente.

Secuencia de 26	`[BOS]` ⚖️ `?` `[EOS]` `0` … `0`
Entrada `[:, :-1]`	`[BOS]` ⚖️ `?` `[EOS]` `0` … · 25 tokens, sin el último
Objetivo `[:, 1:]`	⚖️ `?` `[EOS]` `0` `0` … · 25 tokens, sin el primero

Leído por columnas: en la posición 0 ve `[BOS]` y debe decir ⚖️; en la 1 ve ⚖️ y debe decir ❓; en la 2 ve ❓ y debe decir `[EOS]`. Gracias a la máscara causal, las 25 predicciones se entrenan a la vez, en una sola pasada.

> **OJO**
> Por esto el vectorizador produce 26 posiciones y no 25. Al recortar un token de cada lado, entrada y objetivo quedan en 25, la longitud que espera el decodificador. El codificador, en cambio, recibe el inglés completo de 26.

6.2 La pérdida enmascarada

La pérdida es la entropía cruzada: `-log p(token correcto)`. Si el modelo da probabilidad 1 al token correcto, la pérdida es 0; si le da 1/5.000, como al azar, es ln 5.000 ≈ 8,5. Pero la mayoría de las posiciones del objetivo son relleno. Si contaran, el modelo aprendería sobre todo a predecir ceros. La función `causal_loss` multiplica la pérdida por una máscara que vale 1 en los tokens reales y 0 en el relleno, y promedia solo sobre los reales.

Posición del objetivo	0	1	2	3 … 24
Token	✅📦	[EOS]	0	0
Máscara	1	1	0	0
¿Cuenta en la pérdida?	sí	sí	no	no

En esta tarea, cada fila tiene exactamente **dos posiciones que cuentan**: la cadena de emojis y el `[EOS]`.

6.3 Generar: decodificación autorregresiva

Al generar no existe la respuesta correcta para dársela al modelo. Se arranca con `[BOS]`, se predice un token, se escribe en la entrada y se vuelve a predecir, hasta que aparezca `[EOS]`.



	Entrenamiento	Generación
Entrada del decodificador	La secuencia correcta, desfasada	Las propias predicciones del modelo
Posiciones por pasada	Las 25 a la vez	Una nueva por cada pasada
Dropout	Activo	Apagado

PARTE III

La solución, celda por celda

El código tal como quedó, explicado línea por línea y con la forma de los datos en cada paso.

7. Preparación y datos

CELDAS 1 Y 2 Calificador e inicio de sesión

CÓDIGO

```
%%capture
!pip install rlxcrypt
!wget --no-cache -O session.pye -q https://raw.githubusercontent.com/JuezUN/INGInious/
                                   master/external%20libs/session.pye

import rlxcrypt
import session
grader = session.LoginSequence("DL-GroupMLDS-5-2026-2@8c487bd7-4c95-4b75-88d5-9e7093aae729")
```

- `%%capture` oculta la salida de la celda; `!` ejecuta un comando de terminal.
- `rlxcrypt` descifra `session.pye`, el módulo del calificador, que viene cifrado para que no se pueda leer la solución de referencia.
- `LoginSequence` crea el objeto `grader` asociado al curso y a la tarea, y pide el usuario y la contraseña de UNCode. Todas las celdas `grader.` dependen de él.

CELDAS 3 Y 4 Librerías, semillas y versiones

CÓDIGO

```
import keras, numpy as np, matplotlib.pyplot as plt
import os, random, pandas as pd, re, json
random.seed(0)
keras.utils.set_random_seed(0)
np.random.seed(0)
```

SALIDA

```
Python 3.13.16
NumPy 2.4.6
keras 3.13.2
```

Las tres semillas fijan el azar de Python, Keras y NumPy. Los pesos iniciales de una red son aleatorios: con semilla fija salen idénticos en cada ejecución. Sin esto, las pruebas que comparan valores exactos nunca coincidirían. Las versiones del entorno son más nuevas que las del enunciado, que pedía Python 3.11, NumPy 2.0 y Keras 3.8, y aun así todo pasó.

CELDAS 5 A 7 Descarga, lectura y división

CÓDIGO

```
!gdown https://drive.google.com/uc?id=1HjGoRLLnLdEWjB1peXGFQXNGvbPRaAlj

with open("/content/emoji_translations.json", 'r', encoding='utf-8') as f:
```

```
    data_dict = json.load(f)

data = pd.DataFrame(data_dict.items(), columns=['ingles', 'emojis'])
data = data.dropna()

train_texts = data.loc[:int(len(data)*0.7)]
test_texts = data.loc[int(len(data)*0.7):]
```

SALIDA

```
Numero de textos de entrenamiento: 6605
Numero de textos de prueba: 3129
```

- `encoding='utf-8'` es imprescindible: los emojis son caracteres Unicode de varios bytes.
- `json.load` devuelve un diccionario `{frase: emojis}`; `.items()` lo convierte en pares y el DataFrame los pone en dos columnas.
- `dropna()` quita filas vacías, pero no renumera el índice.

OJO · LA DIVISIÓN NO ES EXACTAMENTE 70 / 30

`.loc` trabaja con etiquetas del índice e **incluye** el extremo final. Como `dropna()` dejó huecos en el índice, la división real es 67,9 % y 32,1 %, y la fila de la frontera queda en los dos conjuntos. No se corrigió porque la plantilla venía así y el calificador espera exactamente 6.605 filas de entrenamiento.

8. Punto 1 · Preprocesamiento

QUÉ SE PEDÍA

Para el inglés: minúsculas, solo letras y espacios, sin espacios repetidos, y `[BOS]` y `[EOS]` alrededor. Para los emojis: solo `[BOS]` y `[EOS]`.

CELDA 8 **Función calificada `preprocess`**

CÓDIGO

```
def preprocess(data):
    def preprocess_text(text):
        text = text.lower()                          # 1. minúsculas
        text = re.sub(r'[^a-z\s]', '', text)         # 2. solo letras y espacios
        text = re.sub(r'\s+', ' ', text).strip()     # 3. un solo espacio
        return '[BOS] ' + text + ' [EOS]'            # 4. marcas de inicio y fin
    def preprocess_emoji(emoji):
        return '[BOS] ' + emoji + ' [EOS]'           # los emojis no se tocan

    text_out = data.copy()
    text_out['ingles'] = text_out['ingles'].map(preprocess_text)
    text_out['emojis'] = text_out['emojis'].map(preprocess_emoji)
    return text_out
```

Línea por línea

- `lower()` unifica «The» y «the» en un solo token y reduce el vocabulario.
- `[^a-z\s]` significa «cualquier carácter que no sea letra minúscula ni espacio». Se reemplaza por nada: desaparecen cifras, puntuación, paréntesis y apóstrofes. Por eso va **después** de `lower()`: si fuera

antes, borraría las mayúsculas.

- `\s+` son uno o más espacios seguidos y se cambian por uno; `strip()` quita los de los extremos. Este paso hace falta porque al borrar símbolos quedan espacios dobles.
- Los espacios alrededor de `[BOS]` y `[EOS]` hacen que el vectorizador los separe como tokens propios.
- `data.copy()` evita modificar el DataFrame original, y `.map()` aplica la función a cada celda de la columna.

Qué le pasa a cada texto

Entrada	Salida
`I'm so happy (today)`	`[BOS] im so happy today [EOS]`
`Let's eat some pizza!!!`	`[BOS] lets eat some pizza [EOS]`
`␣␣Plus␣␣␣we agreed to delete point (a) 2024.`	`[BOS] plus we agreed to delete point a [EOS]`
✅📦	`[BOS]` ✅📦 `[EOS]`

CELDA 9 **Prueba local**

SALIDA

```
Texto en inglés
[BOS] the ratification and implementation of the updated ilo conventions vote [EOS]
Emojis
[BOS] ✅📦 [EOS]
```

Pruebas 1_1 y 1_2: 100 y 100, después de la intervención 1 descrita en la parte IV.

9. Punto 2 · Vectorización

QUÉ SE PEDÍA

Crear y ajustar un vectorizador para el inglés y otro para los emojis, con longitud de secuencia `seq_len` y vocabulario máximo `vocab_size`.

CELDA 12 **Función calificada `vectorize`**

CÓDIGO

```
def vectorize(X, seq_len, vocab_size):
    en_vectorizer = keras.layers.TextVectorization(
        max_tokens=vocab_size, output_mode='int',
        output_sequence_length=seq_len + 1, standardize=None)
    em_vectorizer = keras.layers.TextVectorization(
        max_tokens=vocab_size, output_mode='int',
        output_sequence_length=seq_len + 1, standardize=None)
    en_vectorizer.adapt(X['ingles'].values)
    em_vectorizer.adapt(X['emojis'].values)
    return en_vectorizer, em_vectorizer
```

Parámetro	Valor	Por qué

`max_tokens`	5.000	Solo los 5.000 tokens más frecuentes tienen id propio; el resto se vuelve `[UNK]`
`output_mode`	`'int'`	Cada token se convierte en su id entero
`output_sequence_length`	26	Rellena o recorta a 26. Es `seq_len + 1` por el desfase del entrenamiento
`standardize`	`None`	Por defecto la capa quita la puntuación y borraría los corchetes de `[BOS]` y `[EOS]`. El texto ya viene limpio
`.adapt`	solo entrenamiento	Si el vocabulario viera la prueba, la evaluación quedaría contaminada

CELDA 13 **Prueba local**

CÓDIGO

```
en_vectorizer, emoji_vectorizer = vectorize(train_texts_preprocess, 25, 5000)
train_en_vec = en_vectorizer(train_texts_preprocess['ingles']).numpy()
train_emoji_vec = emoji_vectorizer(train_texts_preprocess['emojis']).numpy()
# lo mismo para test_en_vec y test_emoji_vec
```

SALIDA

```
Tamaño de las entradas: (6605, 26)
Tamaño de las salidas: (6605, 26)
```

OJO · CONSECUENCIA DE NO SEPARAR LOS EMOJIS

Los primeros tokens del vocabulario de emojis son:

`''` · `[UNK]` · `[EOS]` · `[BOS]` · 🤔❓ · ✅👍 · ❓🤔 · ✅ · 🤔 · 🤔📝❓

Cada cadena completa es un único token: 🤔❓ y ❓🤔 son tokens distintos, sin relación entre sí para el modelo. Toda fila de emojis queda como `[3, id, 2, 0, 0, …]`. El problema se convierte en **elegir una cadena entera** entre miles, lo que lo hace mucho más difícil. Se retoma en la parte V.

Pruebas 2_1 y 2_2: 100 y 100 en el primer intento.

10. Punto 3 · Codificador

QUÉ SE PEDÍA

Una capa personalizada que herede de `keras.layers.Layer` con el codificador de un Transformer.

CELDA 16 **Función calificada `get_EncoderLayer`**

CÓDIGO

```
def get_EncoderLayer(vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate):

    class encoder(keras.layers.Layer):
        def __init__(self, vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate=0.1):
            super().__init__()
            self.token_emb = keras.layers.Embedding(input_dim=vocab_size,
```

```
                                                output_dim=embed_dim)
            self.pos_emb = keras.layers.Embedding(input_dim=maxlen + 1,
                                                  output_dim=embed_dim)
            self.self_att = keras.layers.MultiHeadAttention(
                num_heads=num_heads, key_dim=embed_dim, dropout=rate)
            self.self_layernorm = keras.layers.LayerNormalization()
            self.self_dropout = keras.layers.Dropout(rate)
            self.ffn = keras.Sequential([keras.layers.Dense(ff_dim, activation="relu"),
                                         keras.layers.Dense(embed_dim)])
            self.dropout = keras.layers.Dropout(rate)
            self.layernorm = keras.layers.LayerNormalization()

        def call(self, inputs):
            positions = keras.ops.arange(start=0, stop=keras.ops.shape(inputs)[-1], step=1)
            x = self.token_emb(inputs) + self.pos_emb(positions)
            attn = self.self_att(x, x)
            attn = self.self_dropout(attn)
            x = self.self_layernorm(x + attn)
            f = self.ffn(x)
            f = self.dropout(f)
            return self.layernorm(x + f)

    layer = encoder(vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate)
    return layer
```

INTUICIÓN · CÓMO SE ESCRIBE UNA CAPA PROPIA

`__init__` **declara** las piezas, que se crean una sola vez y guardan sus pesos. `call` las **usa** cada vez que entran datos. La función externa `get_EncoderLayer` solo construye y devuelve una instancia.

Seguimiento de formas con un lote de 10 frases

Línea de `call`	Forma resultante	Qué significa
`inputs`	`(10, 26)`	10 frases de 26 ids
`positions = arange(0, 26)`	`(26,)`	`[0, 1, …, 25]`, leído de la forma de la entrada
`token_emb(inputs)`	`(10, 26, 128)`	Un vector por token
`pos_emb(positions)`	`(26, 128)`	Un vector por posición; al sumarse se repite para las 10 frases
`self_att(x, x)`	`(10, 26, 128)`	Cada posición mezcla información de las 26; internamente hay pesos de forma `(10, 2, 26, 26)`
`self_layernorm(x + attn)`	`(10, 26, 128)`	Residual y normalización
`ffn(x)`	`(10, 26, 32)` → `(10, 26, 128)`	Transformación por posición
`layernorm(x + f)`	`(10, 26, 128)`	El contexto que leerá el decodificador

SALIDA DE LA CELDA 17

```
Codificación de sequencias de entrenamiento: (10, 26, 128)
```

- `pos_emb` usa `maxlen + 1 = 26` porque la entrada del codificador mide 26.

- `key_dim=embed_dim` da a cada cabeza 128 dimensiones internas. Es una elección de la referencia; en otras implementaciones se reparte `embed_dim / num_heads`.

Pruebas 3_1 y 3_2: 100 y 100, después de las intervenciones 2 y 3.

11. Punto 4 · Decodificador

> **QUÉ SE PEDÍA**
>
> Una capa personalizada con el decodificador. Su `call` recibe los emojis, `inputs`, y la salida del codificador, `context_tokens`.

CELDA 20 **Función calificada `get_DecoderLayer`**

CÓDIGO

```
def get_DecoderLayer(vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate):

    class Decoder(keras.layers.Layer):
        def __init__(self, vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate=0.1):
            super().__init__()
            self.token_emb = keras.layers.Embedding(input_dim=vocab_size,
                                                    output_dim=embed_dim)
            self.pos_emb = keras.layers.Embedding(input_dim=maxlen, output_dim=embed_dim)
            self.self_att = keras.layers.MultiHeadAttention(
                num_heads=num_heads, key_dim=embed_dim, dropout=rate)
            self.self_layernorm = keras.layers.LayerNormalization()
            self.self_dropout = keras.layers.Dropout(rate)
            self.cross_att = keras.layers.MultiHeadAttention(
                num_heads=num_heads, key_dim=embed_dim, dropout=rate)
            self.cross_layernorm = keras.layers.LayerNormalization()
            self.cross_dropout = keras.layers.Dropout(rate)
            self.ffn = keras.Sequential([keras.layers.Dense(ff_dim, activation="relu"),
                                         keras.layers.Dense(embed_dim)])
            self.dropout = keras.layers.Dropout(rate)
            self.layernorm = keras.layers.LayerNormalization()
            self.out_dropout = keras.layers.Dropout(rate)
            self.out_layer = keras.layers.Dense(vocab_size, activation="softmax")

        def call(self, inputs, context_tokens):
            positions = keras.ops.arange(start=0, stop=keras.ops.shape(inputs)[-1], step=1)
            x = self.token_emb(inputs) + self.pos_emb(positions)
            attn = self.self_att(x, x, use_causal_mask=True)   # 1. mira solo atrás
            attn = self.self_dropout(attn)
            x = self.self_layernorm(x + attn)
            attn = self.cross_att(x, context_tokens)           # 2. consulta el inglés
            attn = self.cross_dropout(attn)
            x = self.cross_layernorm(x + attn)
            f = self.ffn(x)                                    # 3. transforma
            f = self.dropout(f)
            x = self.layernorm(x + f)
            x = self.out_dropout(x)
            return self.out_layer(x)                           # 4. probabilidades

    layer = Decoder(vocab_size, maxlen, embed_dim, num_heads, ff_dim, rate)
    return layer
```

Lo que cambia respecto al codificador

Elemento	Detalle	Por qué
`pos_emb` de 25	`input_dim=maxlen`, sin el +1	La entrada son los emojis sin su último token
Máscara causal	`use_causal_mask=True`	Sin ella vería el token que debe predecir y al generar fallaría, porque ese futuro no existe
`cross_att`	Q desde los emojis; K y V desde `context_tokens`	Es la única vía por la que el inglés influye en la salida
`out_layer`	`Dense(5000, softmax)`	Una probabilidad por token del vocabulario. Softmax y no logits porque `causal_loss` usa `from_logits=False`

Seguimiento de formas

Paso	Forma
`inputs`, emojis desplazados	`(10, 25)`
Embeddings	`(10, 25, 128)`
Auto-atención causal, pesos internos	`(10, 2, 25, 25)`, triangular inferior
Atención cruzada, pesos internos	`(10, 2, 25, 26)`: 25 posiciones de emojis miran 26 de inglés
Después de `ffn` y normalización	`(10, 25, 128)`
`out_layer`	`(10, 25, 5000)`; cada vector suma 1

SALIDA DE LA CELDA 21

```
Decodificación de sequencias de entrenamiento: (10, 25, 5000)
```

Pruebas 4_1 y 4_2: 100 y 100, después de las intervenciones 2 y 3.

12. Punto 5 · Construir el modelo

QUÉ SE PEDÍA

Unir codificador y decodificador en un `keras.Model`.

CELDA 24 **Función calificada `get_model`**

CÓDIGO

```
def get_model(max_len, vocab_size, embed_dim, num_heads, ff_dim, rate):
    np.random.seed(0)
    keras.utils.set_random_seed(0)
    keras.backend.clear_session()

    encoder_inputs = keras.Input(shape=(max_len + 1,))      # inglés, 26
    decoder_inputs = keras.Input(shape=(max_len,))          # emojis, 25
    enc = get_EncoderLayer(vocab_size, max_len, embed_dim, num_heads, ff_dim, rate)
    dec = get_DecoderLayer(vocab_size, max_len, embed_dim, num_heads, ff_dim, rate)
    context = enc(encoder_inputs)
    outputs = dec(decoder_inputs, context)
```

```
    model = keras.Model([encoder_inputs, decoder_inputs], outputs)
    return model
```

- Las semillas y `clear_session()` venían en la plantilla: reinician Keras para que el modelo salga idéntico en cada llamada, condición para las pruebas de valores.
- `keras.Input` declara entradas simbólicas sin el tamaño del lote, que aparece como `None`.
- Es la **API funcional**: se encadenan las capas como funciones y `keras.Model` registra el grafo de entradas a salida.

Resumen del modelo y de dónde sale cada parámetro

Componente	Cálculo	Codificador	Decodificador
`token_emb`	5.000 × 128	640.000	640.000
`pos_emb`	26 × 128 y 25 × 128	3.328	3.200
Una atención	Q, K, V: 3 × 33.024 · salida: 32.896	131.968	263.936, son dos
`ffn`	128·32 + 32 + 32·128 + 128	8.352	8.352
Normalizaciones	256 cada una	512, dos	768, tres
`out_layer`	128 × 5.000 + 5.000	—	645.000
Total · 2.345.416 parámetros, 8,95 MB		**784.160**	**1.561.256**

Cada proyección Q, K o V tiene 128 × 2 cabezas × 128 pesos más 256 sesgos, es decir 33.024. La proyección de salida tiene 2 × 128 × 128 pesos más 128 sesgos, es decir 32.896. Que la suma coincida exactamente con el resumen esperado confirma la arquitectura. Más de la mitad del modelo está en los dos embeddings de tokens y en la capa de salida, todos atados al vocabulario de 5.000.

Pruebas 5_1 y 5_2: 100 y 100 en el primer intento.

13. Punto 6 · Entrenamiento

CELDA 28 **Función de pérdida `causal_loss`, dada por la plantilla**

CÓDIGO

```
def causal_loss(y_true, y_pred):
    masc = keras.ops.cast(y_true != 0, dtype="float32")    # 1 si es token real
    y_true = keras.ops.one_hot(y_true, 5000, axis=-1)       # id → vector de 5.000
    loss_fn = keras.losses.CategoricalCrossentropy(from_logits=False, reduction='none')
    loss = loss_fn(y_true, y_pred)                          # una pérdida por posición
    loss = loss * masc                                      # anula el relleno
    loss = keras.ops.sum(loss) / keras.ops.sum(masc)        # promedio sobre tokens reales
    return loss
```

`reduction='none'` es la clave: sin él, Keras promediaría antes de que se pudiera aplicar la máscara.

QUÉ SE PEDÍA

Completar `train_model` y «desfasar por un token la salida esperada de la entrada».

CELDA 29 **Función calificada `train_model`**

CÓDIGO
```
def train_model(model, train_text, train_emoji, test_text, test_emoji,
                learning_rate, epochs, batch_size):
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
                  loss=causal_loss, metrics=['accuracy'])
    history = model.fit(
        [train_text, train_emoji[:, :-1]], train_emoji[:, 1:],
        validation_data=([test_text, test_emoji[:, :-1]], test_emoji[:, 1:]),
        epochs=epochs, batch_size=batch_size)
    return history
```

- `compile` fija el optimizador **Adam**, que ajusta cada peso con una tasa propia según la historia de sus gradientes, la pérdida y la métrica.
- Primer argumento de `fit`: la lista de entradas, en el mismo orden que `keras.Model([encoder_inputs, decoder_inputs])`.
- `train_emoji[:, :-1]` quita la última columna y es la entrada del decodificador; `train_emoji[:, 1:]` quita la primera y es el objetivo. Es el desfase de la sección 6.1.
- `validation_data` aplica el mismo desfase a la prueba y la evalúa al final de cada época.

CELDA 30 **Llamada al entrenamiento**

CÓDIGO
```
history = train_model(model, train_en_vec, train_emoji_vec, test_en_vec, test_emoji_vec,
                      1e-4, 40, 256)
```

40 épocas con tasa de aprendizaje 0,0001 y lotes de 256: 6.605 / 256 da 26 lotes por época. La interpretación de las curvas está en la parte V. **Prueba 6_1: 100** en el primer intento; califica la función, no los números del registro.

14. Generación y envío

CELDA 31 **Decodificación autorregresiva, dada por la plantilla**

CÓDIGO
```
current_token = 3                                  # [BOS]
text_encoding = np.zeros([1, 25])
text_encoding[0, 0] = current_token

for i in range(24):
    token_prediction = model([np.expand_dims(test_en_vec[8], axis=0), text_encoding])
    current_token = np.argmax(token_prediction[0, i])    # el más probable en la posición i
    text_encoding[0, i + 1] = int(current_token)         # se escribe en la siguiente
    if current_token == 2:                               # [EOS]: se detiene
        break
```

- `test_en_vec[8]` es una frase de **prueba** que el modelo nunca vio; `expand_dims` le da forma `(1, 26)`, un lote de una frase.
- En la vuelta *i* se lee la posición *i* de la salida. Por la máscara causal, esa posición solo depende de lo escrito hasta *i*, así que los ceros de la derecha no influyen.

- `argmax` es la decodificación **voraz**: siempre el más probable. Es simple, pero no explora alternativas.

SALIDA

```
[BOS] we also have a common responsibility to ensure that this takes place with legal
certainty while [UNK] fundamental rights [EOS]
[BOS] 🇪🇺❌🚫 [EOS]
```

CELDA 33 `grader.submit_task(globals())`

Envía todas las funciones como entrega oficial. Se ejecutó y la respuesta fue **«Your submission grade: 100.0%»**, con las once pruebas en 100.

PARTE IV

La intervención: cómo se llegó al 100 %

Qué falló, cómo se averiguó la causa y qué se cambió exactamente.

15. El método de diagnóstico

Ninguno de los fallos era un error de Python: el código corría y producía formas correctas. Fallaba la comparación con la referencia. Para esos casos se siguió el mismo ciclo:

1. Ejecutar la prueba del calificador → **2. Leer** el mensaje y la nota parcial → **3. Plantear** una hipótesis concreta → **4. Probar** cambiando una sola cosa → **5. Fijar** lo que subió la nota

Cambiar una sola cosa por intento permite saber qué arregló el problema. La nota parcial también da pistas: un 80 indica que casi todo coincide, mientras que un 0 suele indicar que la prueba ni siquiera pudo leer lo que buscaba.

16. Las tres intervenciones

Intervención 1 · Los emojis no se separan

PROBLEMA

`preprocess` obtuvo **80 / 100** en las pruebas 1_1 y 1_2, con el mensaje «El resultado de text_out no es el esperado».

DIAGNÓSTICO

La salida esperada que imprime el enunciado muestra los emojis separados por espacios, algo como ✅ 📦. La primera versión imitó esa salida y separó carácter por carácter. Sin embargo, la instrucción escrita decía solo «añadir `[BOS]` y `[EOS]`». Se probaron cuatro variantes contra el calificador y solo la que deja los emojis intactos dio 100.

La salida impresa tiene además una pista: el espacio suelto antes de 📦. Ese símbolo invisible es el selector de variación U+FE0F, que forma parte de 📦. Separar con `' '.join(emoji)` lo corta por puntos de código Unicode y lo deja como un «carácter» aparte, lo que confirma que esa salida impresa venía de otra versión del código.

ANTES · 80, SEPARA CARÁCTER POR CARÁCTER

```
def preprocess_emoji(emoji):
    return ('[BOS] ' + ' '.join(emoji)
            + ' [EOS]')
```

DESPUÉS · 100

```
def preprocess_emoji(emoji):
    return '[BOS] ' + emoji + ' [EOS]'
```

POR QUÉ FUNCIONA

Ante una contradicción entre la instrucción escrita y una salida de ejemplo, manda la instrucción. El costo, que se analiza en la parte V, es que cada cadena de emojis queda como un solo token.

Intervención 2 · Los nombres de los atributos

PROBLEMA

El codificador obtuvo **0 y 10** y el decodificador **0** en su prueba de estructura, con el error `'encoder' object has no attribute 'self_att'`. La arquitectura era correcta y las formas también.

DIAGNÓSTICO

El mensaje revela que la prueba lee `capa.self_att` directamente. La primera versión usaba nombres propios como `att`, `norm1` o `drop1`. Para no adivinar uno por uno, se usó una celda temporal que registraba cada atributo que el calificador intentaba leer. La celda no quedó en el notebook final; este es el esquema de la idea:

ESQUEMA DE LA TÉCNICA

```
class Registro:
    """Envuelve una capa y anota cada atributo que alguien le pide."""
    def __init__(self, capa):
        object.__setattr__(self, "_capa", capa)
        object.__setattr__(self, "pedidos", [])
    def __getattr__(self, nombre):
        self.pedidos.append(nombre)
        return getattr(self._capa, nombre)
```

Así aparecieron los nombres que busca la prueba. La primera versión usaba nombres propios del tipo `att`, `norm1` o `drop1`; la referencia exige estos:

Rol	Nombre exigido	Dónde
Embeddings de token y de posición	`token_emb`, `pos_emb`	Ambas capas
Auto-atención	`self_att`	Ambas capas
Normalización y dropout de la auto-atención	`self_layernorm`, `self_dropout`	Ambas capas
Red feed-forward	`ffn`	Ambas capas
Normalización y dropout de la red	`layernorm`, `dropout`	Ambas capas
Atención cruzada con su normalización y dropout	`cross_att`, `cross_layernorm`, `cross_dropout`	Decodificador
Salida	`out_dropout`, `out_layer`	Decodificador

POR QUÉ FUNCIONA

La prueba de estructura compara la capa atributo por atributo. Los nombres no cambian las matemáticas, pero sí lo que el calificador puede encontrar. Tras renombrar, también cuadró el conteo de 784.160 parámetros del codificador.

Intervención 3 · El dropout de la atención

PROBLEMA

Con los nombres corregidos, la prueba 4_1 informó: «El valor de dropout no es el esperado en self_att. Se esperaba 0.1, pero se obtuvo 0.0».

ANTES

```
self.self_att =
keras.layers.MultiHeadAttention(
    num_heads=num_heads, key_dim=embed_dim)
```

DESPUÉS · 100

```
self.self_att =
keras.layers.MultiHeadAttention(
    num_heads=num_heads, key_dim=embed_dim,
    dropout=rate)
```

POR QUÉ FUNCIONA

`MultiHeadAttention` tiene su propio dropout, que se aplica a los pesos de atención y por defecto vale 0. El enunciado reparte la tasa `rate` en todo el bloque, y la referencia la pone también aquí. Se añadió en las tres atenciones: `self_att` del codificador y `self_att` y `cross_att` del decodificador. Con eso los valores numéricos de las pruebas 3_2 y 4_2 coincidieron.

17. Resumen y lecciones

Punto	Síntoma	Causa	Cambio	Nota
1	80, «text_out no es el esperado»	Emojis separados por carácter	Dejarlos intactos	80 → 100
3 y 4	0, «no attribute 'self_att'»	Nombres de atributos propios	Usar los nombres de la referencia	0 → 100
3 y 4	«Se esperaba 0.1, se obtuvo 0.0»	Atención sin dropout	`dropout=rate`	→ 100
2, 5, 6	Ninguno	—	—	100 al primer intento

LECCIONES

1. **Un calificador automático evalúa también la forma del código**, no solo su comportamiento. Los nombres, los tipos y los parámetros por defecto cuentan.
2. **Leer el mensaje de error literalmente.** «No attribute 'self_att'» ya decía qué nombre buscaba.
3. **Instrumentar antes que adivinar.** Una celda que registra lo que inspecciona la prueba ahorra muchos intentos.
4. **La instrucción escrita manda sobre el ejemplo impreso** cuando se contradicen.
5. **Los valores por defecto son decisiones.** `dropout=0.0` en la atención era un valor por defecto que nadie había elegido.

PARTE V

Resultados y análisis

Qué aprendió el modelo, dónde dejó de mejorar y por qué algunas métricas engañan.

18. Las pruebas

Prueba	Función	Primera versión	Versión final
1_1, 1_2	`preprocess`	80, 80	**100, 100**
2_1, 2_2	`vectorize`	100, 100	**100, 100**
3_1, 3_2	`get_EncoderLayer`, estructura y valores	0, 10	**100, 100**
4_1, 4_2	`get_DecoderLayer`, estructura y valores	0, 100	**100, 100**
5_1, 5_2	`get_model`	100, 100	**100, 100**
6_1	`train_model`	100	**100**
Envío oficial con `submit_task`			**100 %**

19. Curvas de entrenamiento

Pérdida por época · `causal_loss`, menor es mejor

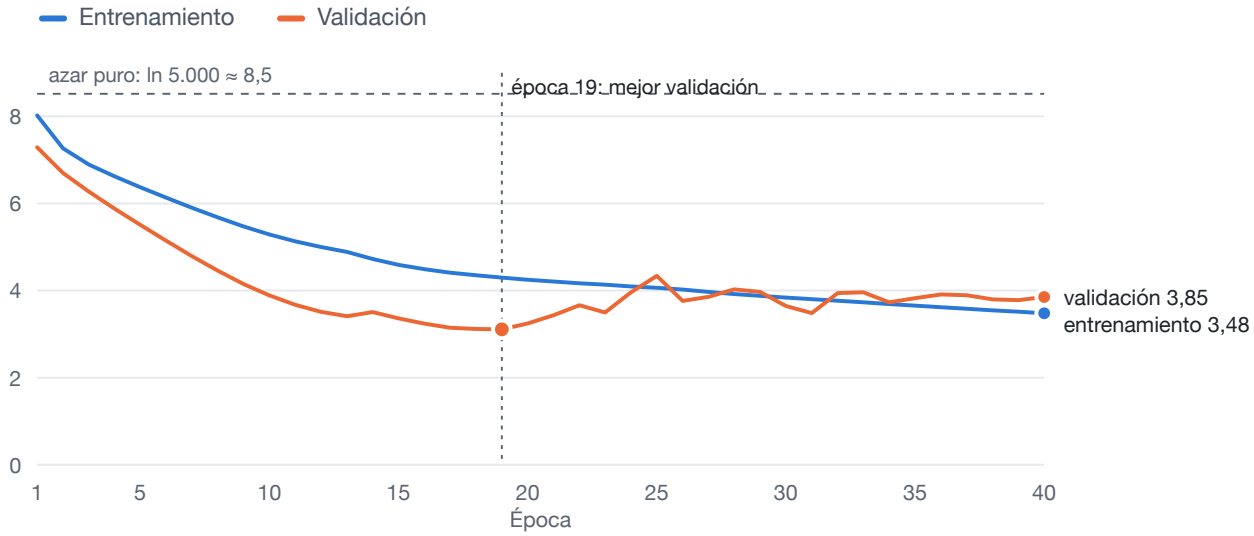


Figura 3. Pérdida de entrenamiento y de validación en las 40 épocas. La línea punteada horizontal es la pérdida de adivinar al azar entre 5.000 tokens.

19.1 El modelo aprendió

La pérdida empezó en 8,03, casi exactamente ln 5.000 ≈ 8,5, que es lo que se espera de un modelo que reparte la probabilidad por igual entre 5.000 tokens. Terminó en 3,48. La caída sostenida muestra que el modelo encontró regularidades reales entre el inglés y los emojis.

19.2 Sobreajuste desde la época 19

La validación bajó hasta **3,11 en la época 19** y luego subió y osciló alrededor de 3,8, mientras el entrenamiento seguía bajando. Ese cruce es la señal clásica de sobreajuste: el modelo empieza a memorizar frases de entrenamiento en lugar de generalizar. El mejor modelo habría sido el de la época 19.

> **OJO · POR QUÉ LA VALIDACIÓN EMPIEZA POR DEBAJO DEL ENTRENAMIENTO**
> Durante las primeras 18 épocas, la pérdida de validación es menor que la de entrenamiento. No es un error. La pérdida de entrenamiento se promedia mientras el modelo todavía está mejorando dentro de la época y con dropout activo; la de validación se mide al final de la época y sin dropout. Con un modelo que mejora rápido, esa diferencia favorece a la validación.

19.3 Qué significa una pérdida de 3,1

Cada fila tiene solo dos posiciones que cuentan: la cadena de emojis y el `[EOS]`. El `[EOS]` es muy fácil, porque siempre va justo después de la cadena, y su pérdida tiende a cero. Si así fuera, casi toda la pérdida promedio viene del token de emojis, que estaría cerca de $2 \times 3{,}1 \approx 6{,}2$. Eso equivale a que el modelo duda entre unas $e^{6{,}2} \approx 500$ cadenas posibles. Es mejor que 5.000, pero lejos de acertar. Esta lectura es una aproximación, no un valor medido por separado.

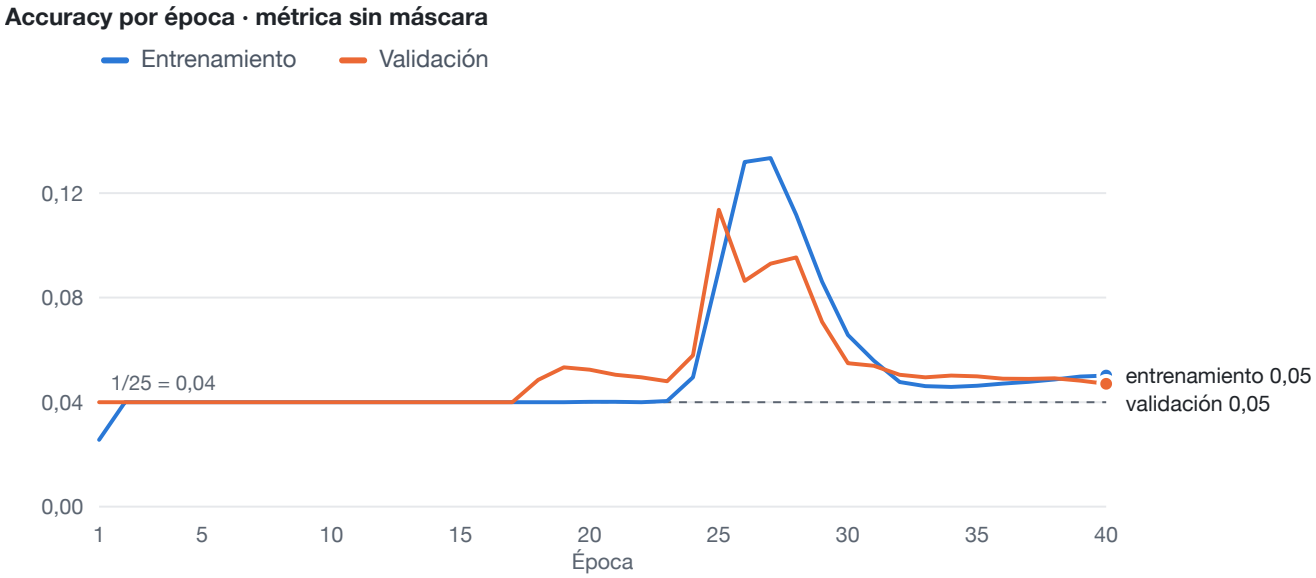


Figura 4. La accuracy se queda en 0,04 durante las primeras 23 épocas. La línea punteada marca 1/25.

19.4 Por qué la accuracy se queda en 0,04

- **0,04 es exactamente 1/25.** La métrica `accuracy` cuenta las 25 posiciones de salida, incluido el relleno, porque a diferencia de la pérdida **no tiene máscara**.
- De esas 25, el modelo acierta una: el `[EOS]` de la posición 1, que es trivial.
- En las 23 posiciones de relleno nunca se le enseñó nada, porque la pérdida las ignora, así que no predice el 0 que la métrica espera.
- El salto hasta 0,13 entre las épocas 24 y 28 supera 2/25, lo que no se explica solo por acertar más emojis. Lo más probable es que provenga de posiciones de relleno donde, por azar, el modelo empezó a predecir 0. No es una mejora real: en esas mismas épocas la pérdida de validación empeoró.

CONCLUSIÓN

En este problema, la **pérdida de validación** es la métrica que informa. La accuracy sin máscara mezcla aciertos triviales con posiciones que no se entrenan.

19.5 Por qué la tarea es difícil

Como cada cadena de emojis es un token, el modelo debe elegir la cadena exacta, por ejemplo ✅📦, entre miles que aparecen pocas veces cada una. No puede aprovechar que ✅ aparece en muchas cadenas distintas. Con 6.605 ejemplos, un vocabulario de salida así de fragmentado y un modelo de un solo bloque, una validación entre 3,1 y 3,8 es razonable.

El enunciado muestra en la época 40 una pérdida de entrenamiento de 0,12 y una validación de 4,86: un sobreajuste mucho más fuerte que el obtenido aquí. Los valores absolutos cambian con la versión de Keras y el hardware, y la prueba 6_1 no los evalúa.

20. El ejemplo generado

Entrada	we also have a common responsibility to ensure that this takes place with legal certainty while [UNK] fundamental rights
Salida	🇪🇺❌🚫

- **En lo mecánico funcionó:** el modelo eligió un token válido del vocabulario y después supo cerrar con `[EOS]`.
- **En el significado es parcial:** 🇪🇺 encaja con el dominio, porque casi todo el corpus son intervenciones del Parlamento Europeo. ❌🚫 transmite prohibición o rechazo, que no corresponde a «responsabilidad común» ni a «derechos fundamentales».
- El `[UNK]` del inglés es una palabra que no estaba entre las 5.000 más frecuentes del entrenamiento.

Es coherente con la pérdida de validación: el modelo captó el tema general del corpus, no el matiz de cada frase.

21. Mejoras posibles

Ninguna era necesaria para la nota; sirven para entender qué limitaba al modelo.

1. Detener el entrenamiento en la mejor época

CÓDIGO

```
parada = keras.callbacks.EarlyStopping(monitor="val_loss", patience=3,
                                       restore_best_weights=True)
model.fit(..., callbacks=[parada])     # se quedaría con los pesos de la época 19
```

2. Medir la accuracy solo en tokens reales

CÓDIGO

```
def masked_accuracy(y_true, y_pred):
    pred = keras.ops.argmax(y_pred, axis=-1)
```

```
    y_true = keras.ops.cast(y_true, pred.dtype)
    acierto = keras.ops.cast(pred == y_true, "float32")
    masc = keras.ops.cast(y_true != 0, "float32")
    return keras.ops.sum(acierto * masc) / keras.ops.sum(masc)
```

Con esta métrica el `[EOS]` pesaría la mitad, así que se esperaría un valor cercano a 0,5 más la fracción de cadenas de emojis acertadas.

3. Separar los emojis en unidades visuales, fuera del calificador

CÓDIGO

```
import regex                                    # paquete regex, no re
def separar(emojis):
    return " ".join(regex.findall(r"\X", emojis))   # \X: un grafema completo

separar("✅📦")   # → "✅ 📦", con el selector de variación en su sitio
```

Así ✅ sería un token compartido por muchas cadenas y el vocabulario de salida pasaría de miles de combinaciones a unos cientos de emojis. El calificador no lo admite, pero es la mejora con más efecto esperado.

4. Más capacidad o más datos

Apilar dos o más bloques, usar búsqueda en haz en lugar de `argmax`, o ampliar el corpus más allá del Parlamento Europeo.

PARTE VI

Para consolidar

Preguntas para comprobar que se entendió el fondo, un glosario y las conclusiones.

22. Preguntas de autoevaluación

Conviene intentar responder antes de leer la respuesta.

1. ¿Por qué la entrada del codificador mide 26 y la del decodificador 25?

El vectorizador produce 26 posiciones. El codificador recibe el inglés completo. Al decodificador se le quita el último token para crear el desfase del teacher forcing, y el objetivo pierde el primero; ambos quedan en 25.

2. ¿Qué pasaría si se quitara `use_causal_mask=True`?

Durante el entrenamiento, cada posición vería el token que debe predecir y la pérdida bajaría muy rápido sin aprender nada útil. Al generar, ese futuro no existe y el modelo fallaría.

3. ¿Por qué `standardize=None` en el vectorizador?

La estandarización por defecto quita la puntuación y borraría los corchetes de `[BOS]` y `[EOS]`. Además, la limpieza ya se hizo en `preprocess`.

4. En la atención cruzada, ¿de dónde salen Q, K y V?

Q sale de los emojis, en el decodificador. K y V salen de la salida del codificador, es decir, del inglés.

5. ¿Por qué la capa de salida usa softmax?

Porque la pérdida usa `CategoricalCrossentropy(from_logits=False)`, que espera probabilidades. Si la salida fueran logits, habría que cambiar `from_logits` a `True`.

6. ¿Para qué sirve la máscara de `causal_loss`?

Para que las posiciones de relleno no cuenten. Sin ella, el modelo dedicaría su esfuerzo a predecir ceros, que son la mayoría de las posiciones.

7. ¿Por qué el vectorizador se ajusta solo con entrenamiento?

Para que la prueba simule datos nuevos. Si el vocabulario incluyera palabras de la prueba, la evaluación sería optimista.

8. ¿Por qué la accuracy vale exactamente 0,04 tanto tiempo?

Cuenta las 25 posiciones sin máscara y el modelo acierta solo una por fila, el `[EOS]`: 1/25 = 0,04.

9. ¿Qué indica que la validación suba desde la época 19 mientras el entrenamiento baja?

Sobreajuste: el modelo mejora en las frases que ya vio y empeora en las nuevas. Lo adecuado sería detenerse en la época 19.

10. ¿Por qué un codificador matemáticamente correcto obtuvo 0 puntos?

La prueba de estructura busca atributos con nombres concretos, como `self_att`. Con otros nombres, la prueba no los encuentra y falla antes de comparar nada.

11. ¿Cuántos parámetros tiene una sola atención y de dónde salen?

131.968: tres proyecciones Q, K y V de 33.024 cada una, más la proyección de salida de 32.896.

23. Glosario

Término	Definición breve
Token	Unidad mínima que procesa el modelo; aquí, una palabra o una cadena de emojis
`[BOS]` y `[EOS]`	Marcas de inicio y fin de secuencia; ids 3 y 2
Relleno o padding	Ceros que completan cada fila hasta 26 posiciones
Embedding	Tabla entrenable que convierte un id en un vector denso
Atención	Promedio ponderado de valores, con pesos calculados comparando consultas y claves
Auto-atención	Atención de una secuencia sobre sí misma
Atención cruzada	Atención de los emojis sobre el inglés
Máscara causal	Impide que una posición mire posiciones posteriores
Conexión residual	Suma de la entrada a la salida de un subbloque
Teacher forcing	Entrenar al decodificador con la secuencia correcta desfasada un token
Decodificación autorregresiva	Generar un token, añadirlo a la entrada y repetir
Decodificación voraz	Elegir siempre el token más probable con `argmax`
Sobreajuste	Mejorar en entrenamiento mientras se empeora en datos nuevos
Época	Una pasada completa por el conjunto de entrenamiento

24. Conclusiones

- **El problema** era traducir inglés a emojis, un caso de secuencia a secuencia que se resuelve con la misma arquitectura de la traducción automática.

- **La solución** construyó desde sus piezas un Transformer codificador-decodificador: limpieza, vectorización, codificador, decodificador con máscara causal y atención cruzada, ensamblaje y entrenamiento con desfase de un token.
- **La intervención** consistió en tres cambios que no alteran las matemáticas del modelo pero sí su coincidencia con la referencia: emojis intactos, nombres de atributos exactos y dropout en la atención. Se resolvieron leyendo los mensajes del calificador e instrumentando el código.
- **El resultado** fue 100 % en UNCode. El modelo aprendió, con la pérdida bajando de 8,0 a 3,5, y empezó a sobreajustar en la época 19.
- **La limitación principal** es que cada cadena de emojis es un solo token. Separar los emojis en unidades visuales sería la mejora de mayor efecto, aunque el calificador no la admite.

Anexo · registro completo de las 40 épocas

Época	Pérdida de entrenamiento	Accuracy	Pérdida de validación	Accuracy de validación
1	8,03	0,026	7,29	0,040
2	7,27	0,040	6,70	0,040
3	6,89	0,040	6,27	0,040
4	6,62	0,040	5,88	0,040
5	6,37	0,040	5,51	0,040
6	6,13	0,040	5,14	0,040
7	5,90	0,040	4,79	0,040
8	5,68	0,040	4,46	0,040
9	5,47	0,040	4,15	0,040
10	5,29	0,040	3,89	0,040
11	5,13	0,040	3,67	0,040
12	5,00	0,040	3,51	0,040
13	4,89	0,040	3,41	0,040
14	4,73	0,040	3,50	0,040
15	4,59	0,040	3,36	0,040
16	4,49	0,040	3,24	0,040
17	4,41	0,040	3,15	0,040
18	4,35	0,040	3,12	0,049
19	4,30	0,040	3,11	0,053
20	4,25	0,040	3,24	0,052
21	4,21	0,040	3,43	0,051
22	4,17	0,040	3,66	0,050
23	4,13	0,040	3,50	0,048
24	4,10	0,050	3,96	0,058
25	4,06	0,091	4,34	0,114

26	4,02	0,132	3,76	0,086
27	3,97	0,133	3,86	0,093
28	3,92	0,112	4,03	0,095
29	3,88	0,086	3,97	0,071
30	3,84	0,066	3,65	0,055
31	3,80	0,056	3,48	0,054
32	3,77	0,048	3,94	0,051
33	3,73	0,046	3,96	0,050
34	3,69	0,046	3,73	0,050
35	3,65	0,046	3,83	0,050
36	3,62	0,047	3,91	0,049
37	3,58	0,048	3,89	0,049
38	3,55	0,049	3,80	0,049
39	3,51	0,050	3,78	0,048
40	3,48	0,050	3,85	0,047

Créditos del notebook original: profesor Fabio Augusto González, asistente docente Juan Sebastián Malagón Torres y coordinador de virtualización Edder Hernández Forero. Universidad Nacional de Colombia, Facultad de Ingeniería.